

Autonomous Docking and Navigation

Battery-Aware Waypoint Patrol with Autonomous Docking

Project Report

Team Name — IRkiPP

Platform: Gazebo Classic + TurtleBot3
Framework: ROS2 Humble + Nav2
Language: Python 3

Name	Roll No.
Vansh Jaiswal	2024102013
Priyanshu Gupta	2024102023
Anuj Kumar Tiwari	2024102021

Platform Ubuntu 22.04 / ROS2 Humble
Simulator Gazebo Classic 11
Robot TurtleBot3

Abstract

This report presents an autonomous mobile robot mission system for the TurtleBot3 under ROS2 Humble and Nav2. The system implements battery-aware waypoint patrol with autonomous docking and predictive scheduling — the robot estimates the energy cost of its remaining path before each departure rather than relying on a fixed battery threshold. Three ROS2 nodes are implemented: a *battery node* simulating velocity-dependent drain and fixed-rate charging; a *waypoint node* for sequential baseline patrol; and a *decision node* running a six-state finite state machine with three-tier path estimation, two-waypoint energy lookahead, smart charge targeting, and PID precision docking.

Contents

1	Introduction	3
2	System Architecture	3
2.1	Inter-Node Communication	3
3	Battery Node	4
3.1	Drain Model	4
3.2	Algorithm	5
4	Waypoint Node	5
4.1	Algorithm	6
5	Decision Node	6
5.1	Finite State Machine	7
5.2	Decision Node Algorithm	8
5.3	Three-Tier Path Length Estimation	8
5.4	Energy Safety Check	9
5.5	Smart Charge Target	9
5.6	PID Fine-Docking	10
5.7	Dock Selection and Orientation	10
6	Visualisation	10
7	Design Decisions	11
8	Results and Discussion	11
8.1	Parameter Tuning During Testing	11
8.2	Map 1 — Gazebo Default Map	12
8.2.1	Gazebo and RViz	12
8.2.2	Terminal Output	13
8.2.3	RQT Graph	13
8.2.4	TF Frame Tree	13
8.2.5	Docking and Battery Trace	14
8.2.6	Problems and Compromises	14
8.3	Map 2 — Custom Map	15
8.3.1	Gazebo and RViz	15

8.3.2 Problems and Compromises	16
9 Conclusion	17

1. Introduction

The project implements a persistent navigation pipeline for a TurtleBot3 operating in a fixed Gazebo environment. The robot simulates a virtual battery that depletes during motion and must autonomously decide when to divert to a charging dock before resuming its mission.

The core requirement — battery-aware waypoint patrol with docking — is fully satisfied. The bonus predictive scheduling requirement is also fulfilled: instead of a fixed threshold, the robot calculates the exact energy cost of the remaining route using Nav2’s path planner and a learned path-overhead ratio, then charges only to the minimum level needed to complete the mission.

2. System Architecture

The system consists of three independent ROS2 nodes communicating through topics and action servers.

Table 1: ROS2 nodes and their roles.

Node	File	Role
battery_node	battery_node.py	Simulates battery drain and charging; publishes /battery_level at 1 Hz.
waypoint_node	waypoint_node.py	Stage 1 baseline: sequential Nav2 patrol, no energy management.
decision_node	decision_node.py	Stage 3 executive: full FSM, energy prediction, PID docking, visualisation.

2.1. Inter-Node Communication

Figure 1 shows the data flow between nodes, the Nav2 stack, and Gazebo.

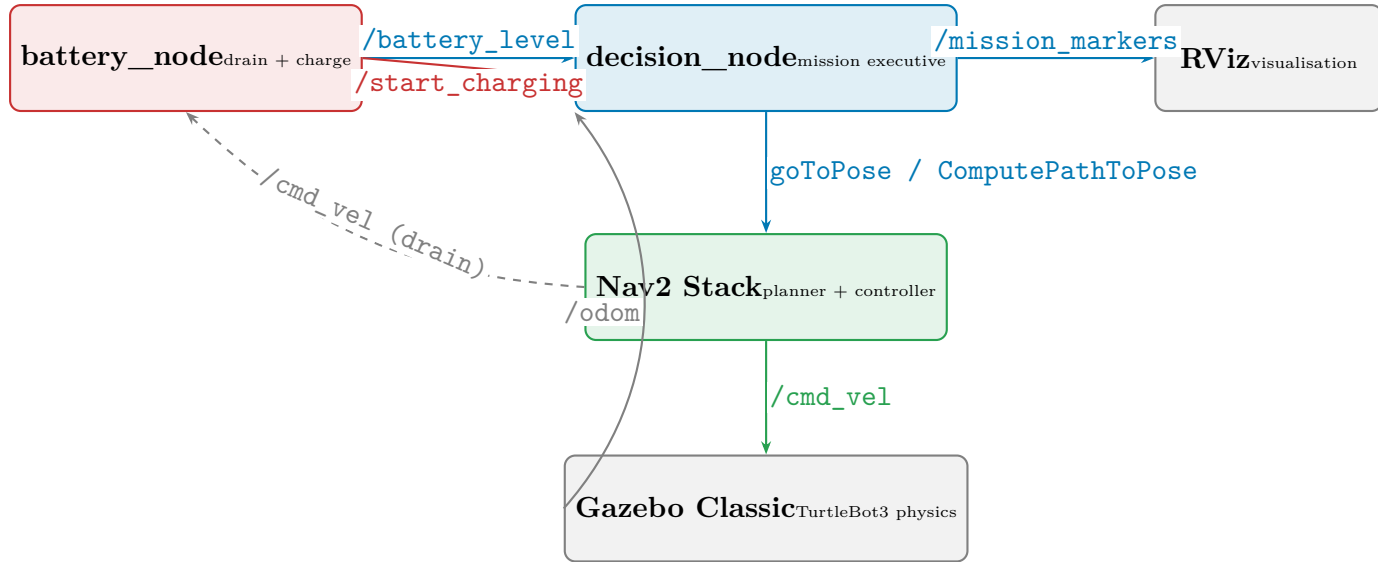


Figure 1: System architecture and inter-node data flow. Dashed arrow shows the `/cmd_vel` feed used by `battery_node` for velocity-dependent drain.

Table 2: Complete ROS2 interface summary.

Topic / Action	Type	Dir.	Purpose
<code>/battery_level</code>	Float32	battery→decision	Current battery % (1 Hz)
<code>/start_charging</code>	Bool	decision→battery	Activate/deactivate charging
<code>/cmd_vel</code>	Twist	Nav2/PID→Gazebo	Robot velocity commands
<code>/odom</code>	Odometry	Gazebo→decision	Robot pose and heading
<code>/mission_markers</code>	MarkerArray	decision→RViz	HUD panel, waypoint/dock markers
<code>/spawn_entity</code>	Service	decision→Gazebo	Spawn sphere markers at startup
<code>goToPose</code>	Action	decision→Nav2	Send navigation goal
<code>ComputePathToPose</code>	Action	decision→Nav2	Silent path-length query

3. Battery Node

`battery_node.py` simulates the robot’s onboard battery. Every second it either drains the battery proportionally to the robot’s instantaneous velocity (from `/cmd_vel`) or adds a fixed charge if `/start_charging` is `True`. The current percentage is published on `/battery_level` at 1 Hz.

3.1. Drain Model

$$\Delta B_{\text{drain}} = v_{\text{lin}} \cdot k_{\text{lin}} + \omega \cdot k_{\text{ang}} \quad (1)$$

where $k_{\text{lin}} = 3.0\% / (\text{m/s})$ and $k_{\text{ang}} = 2.0\% / (\text{rad/s})$. Charging adds $+5.0\% / \text{s}$. Battery is clamped to $[0, 100]$ after every update.

3.2. Algorithm

Algorithm 1 Battery Node — runs at 1 Hz

- 1: Start with battery = 100%, charging = False
 - 2: **On** `/start_charging`: update charging flag
 - 3: **On** `/cmd_vel`: record linear and angular velocity
 - 4: **procedure** EVERY SECOND
 - 5: **if** charging **then** Add 5.0% to battery
 - 6: **else** Subtract $(v_{\text{lin}} \times 3.0 + \omega \times 2.0)$ from battery
 - 7: **end if**
 - 8: Clamp battery to $[0, 100]$; publish on `/battery_level`
 - 9: **end procedure**
-

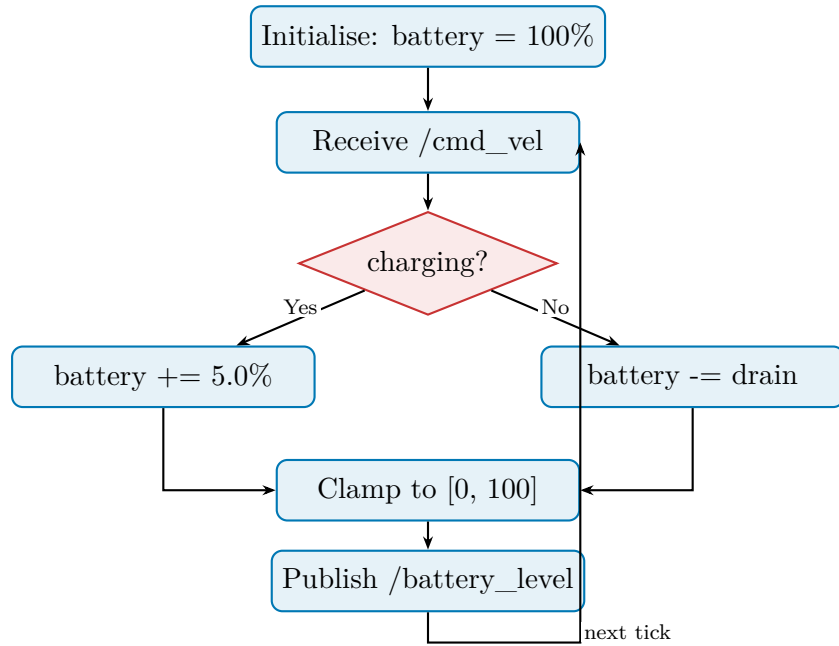


Figure 2: Battery node flowchart.

4. Waypoint Node

`waypoint_node.py` navigates the robot through a list of waypoints using Nav2's `BasicNavigator`. There is no battery awareness or docking — it serves as the Stage 1 baseline.

4.1. Algorithm

Algorithm 2 Waypoint Node — runs at 1 Hz

```

1: Wait for Nav2 active; load waypoint list; set current_index = 0
2: procedure EVERY SECOND
3:   if Nav2 still executing then return
4:   end if
5:   if all waypoints visited then log finished, stop
6:   end if
7:   Send waypoints[current_index] to Nav2; increment index
8: end procedure

```

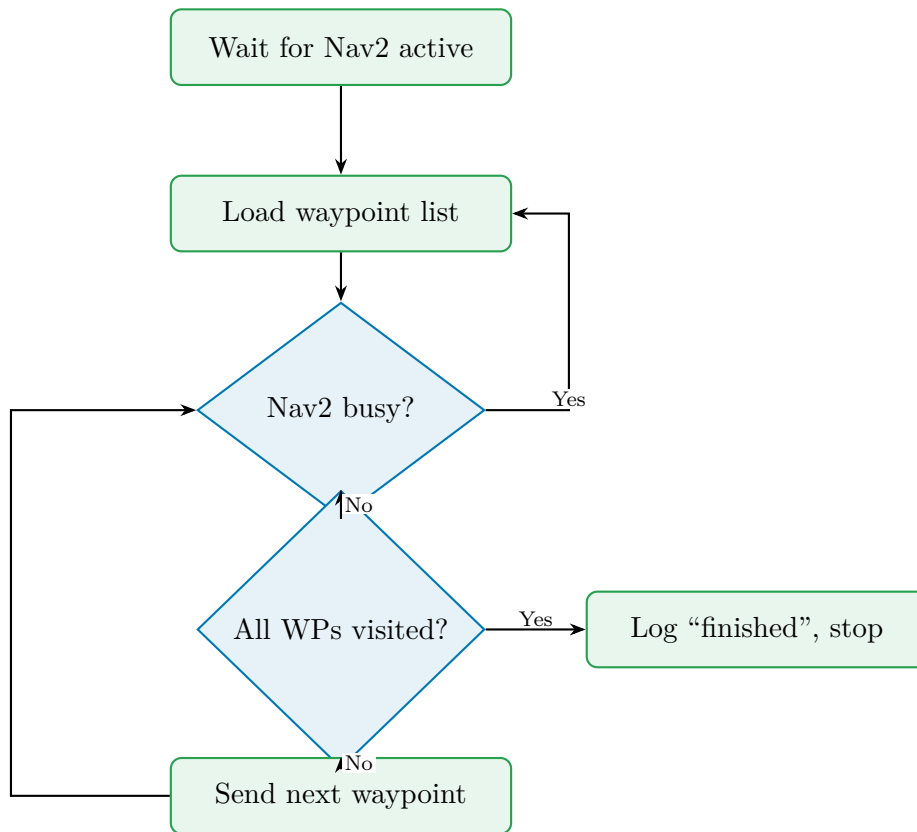


Figure 3: Waypoint node flowchart.

5. Decision Node

`decision_node.py` is the full mission executive. It reads waypoints and dock positions from `config/mission.yaml`, manages a six-state FSM, estimates energy costs, and controls the robot through both Nav2 (coarse navigation) and direct PID velocity commands (precision docking).

5.1. Finite State Machine

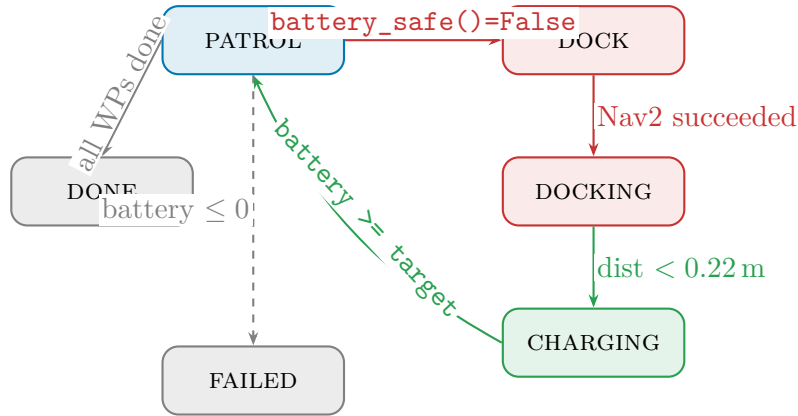


Figure 4: Six-state FSM of the **decision_node**. Dashed arrow can originate from any active state.

Table 3: State descriptions.

State	Description
PATROL	Sends waypoints to Nav2 one by one. Runs energy check before each departure. Skips a waypoint after 3 consecutive Nav2 failures.
DOCK	Navigates to nearest non-blacklisted dock via Nav2. Retries indefinitely.
DOCKING	10 Hz PID takes over for final precision approach (Phase 1: rotate; Phase 2: drive). Completes when distance < 0.22 m.
CHARGING	Robot stationary. Waits until battery reaches computed charge target.
DONE	Terminal. All waypoints visited successfully.
FAILED	Terminal. Battery reached 0%. All motion stopped.

5.2. Decision Node Algorithm

Algorithm 3 Decision Node — main loop at 1 Hz

```

1: Load waypoints and docks from mission.yaml; pre-cache Nav2 paths
2: Set state = PATROL
3: procedure EVERY SECOND
4:   if battery  $\leq$  0% then  $\rightarrow$  FAILED, stop all motion
5:   end if
6:   if state = PATROL then
7:     if Nav2 still moving then wait
8:     end if
9:     if all waypoints visited then  $\rightarrow$  DONE
10:    end if
11:    Compute three energy scenarios:
12:     $E_1 = \text{robot} \rightarrow \text{wp}_i \rightarrow \text{nearest dock to wp}_i$ 
13:     $E_2 = \text{robot} \rightarrow \text{wp}_i \rightarrow \text{wp}_{i+1} \rightarrow \text{nearest dock to wp}_{i+1}$ 
14:     $E_3 = \text{robot} \rightarrow \text{nearest dock to robot}$ 
15:    if battery  $< \max(E_1, E_2, E_3) + 25\%$  then  $\rightarrow$  DOCK
16:    else send  $\text{wp}_i$  to Nav2
17:    end if
18:  else if state = DOCK then
19:    Send nearest non-blacklisted dock to Nav2
20:    if Nav2 succeeds then  $\rightarrow$  DOCKING
21:    end if
22:    if failed 3 times then blacklist dock, try next nearest
23:    end if
24:  else if state = CHARGING then
25:    target = energy(dock  $\rightarrow$  remaining wps  $\rightarrow$  dock) + 25%
26:    if battery  $\geq$  target then  $\rightarrow$  PATROL
27:    end if
28:  end if
29: end procedure
30: procedure EVERY 100 MS (DOCKING ONLY)
31:   Compute distance  $\rho$  and heading error to dock
32:   if |heading error|  $> 0.25$  rad then rotate in place only
33:   else drive forward + correct heading
34:   end if
35:   if  $\rho < 0.22$  m then stop, /start_charging = True,  $\rightarrow$  CHARGING
36:   end if
37: end procedure

```

5.3. Three-Tier Path Length Estimation

Rather than using straight-line distance, the node estimates actual navigated distance through a priority cascade:

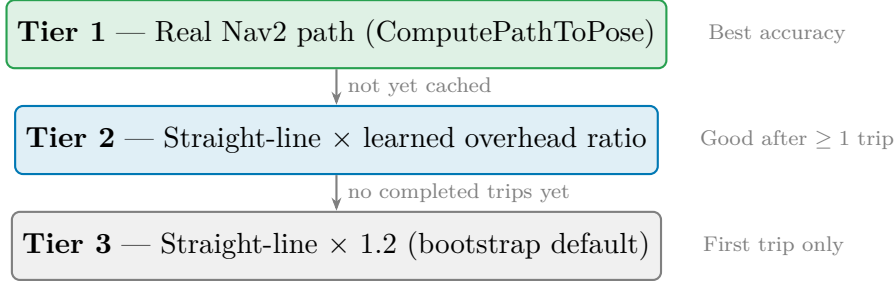


Figure 5: Priority cascade for path-length estimation.

On startup, the node queries Nav2 for real planned paths for every mission segment and stores the results. After each completed trip the node records how much the robot actually travelled versus the straight-line distance and updates a rolling average over the last 10 legs:

$$\hat{r} = \frac{1}{\min(k, 10)} \sum_j \frac{d_{\text{odom},j}}{d_{\text{straight},j}} \quad (2)$$

This learned ratio $\hat{r}$ is used for any segment not yet in the cache (Tier 2). Before any real data exists, the fallback default $\hat{r} = 1.2$ is used (Tier 3).

5.4. Energy Safety Check

All path lengths are converted to battery cost using $\gamma = 4.5\%/m$, the average drain per metre at typical Nav2 speeds. Before every departure for waypoint i , three energy scenarios are checked:

$$E_{\text{wp1}} = \gamma \cdot \text{path}(\text{robot} \rightarrow \text{wp}_i \rightarrow \text{nearest dock to wp}_i) \quad (3)$$

$$E_{\text{wp2}} = \gamma \cdot \text{path}(\text{robot} \rightarrow \text{wp}_i \rightarrow \text{wp}_{i+1} \rightarrow \text{nearest dock to wp}_{i+1}) \quad (4)$$

$$E_{\text{direct}} = \gamma \cdot \text{path}(\text{robot} \rightarrow \text{nearest dock to robot}) \quad (5)$$

The robot departs only if:

$$B > \max(E_{\text{wp1}}, E_{\text{wp2}}, E_{\text{direct}}) + 25\% \quad (6)$$

Each scenario catches a different failure mode. E_{wp1} ensures the robot can reach wp_i and still return to a dock. E_{wp2} catches the stranding trap where the robot reaches wp_i safely but cannot continue to wp_{i+1} or dock from there. E_{direct} is a floor check — the robot must confirm it can still reach a dock from its *current* position before that window closes. Because this check re-runs on every departure it acts as a rolling lookahead.

5.5. Smart Charge Target

Instead of always charging to 100%, the node computes the minimum charge needed for the entire remaining mission:

$$T = \min\left(100, \max(E_{\text{remaining}} + 25\%, 25\%)\right) \quad (7)$$

Table 4: Charge target examples.

Remaining route	$E_{\text{remaining}}$	Target
8 m (3 WPs)	36 %	61 %
0.5 m (1 WP)	2.3 %	27.3 %
20 m (large)	90 %	100 % (capped)
0 m (done)	0 %	25 % (floor)

The 25% margin absorbs path estimation inaccuracy, PID settling overhead, and Nav2 path deviations. The 25% floor when $E_{\text{remaining}} = 0$ ensures the robot never leaves the dock with near-zero battery, preventing a FAILED state during final motion.

5.6. PID Fine-Docking

When Nav2 delivers the robot to within ≈ 0.4 m of the dock, a proportional controller at 10 Hz handles the final approach.

Algorithm 4 PID Docking — runs at 10 Hz in DOCKING state

```

1: Compute distance  $\rho$  and heading error from robot to dock
2: if |heading error| > 0.25 rad then
3:   Rotate in place: angular =  $0.4 \times \text{heading\_error}$ , linear = 0
4: else
5:   Drive: linear =  $\min(0.4 \times \rho, 0.10 \text{ m/s})$ , angular =  $0.8 \times \text{heading\_error}$ 
6: end if
7: Apply smoothing:  $0.8 \times \text{prev} + 0.2 \times \text{new}$  (prevents velocity spikes)
8: Publish on /cmd_vel
9: if  $\rho < 0.22$  m then stop, send /start_charging = True,  $\rightarrow$  CHARGING
10: end if

```

5.7. Dock Selection and Orientation

The robot always picks the nearest dock not in its failure blacklist. If Nav2 fails on that dock 3 times it is blacklisted and the next nearest is tried. If all docks are blacklisted the list clears and selection restarts.

Every Nav2 goal includes a pre-computed arrival heading — at waypoints the robot arrives facing toward wp_{i+1} ; at docks it faces outward to minimise PID Phase 1 rotation. The heading ψ is:

$$\mathbf{q} = (0, 0, \sin(\psi/2), \cos(\psi/2)), \quad \psi = \text{atan2}(\Delta y, \Delta x) \quad (8)$$

6. Visualisation

In **RViz**, all markers are published as `MarkerArray` on `/mission_markers`. Waypoints appear as green cylinders (dimming when visited), docks as orange-red cylinders (greying

when blacklisted), and a floating battery bar follows the robot. A status panel at map coordinates (3.5, 2.5) shows state, battery, waypoint progress, and learned overhead.

In **Gazebo Classic**, sphere models are spawned at startup for waypoints and docks, and a robot-mounted battery indicator sphere shifts from green to amber to red as battery depletes.

7. Design Decisions

Energy estimation over fixed threshold

A fixed dock-trigger threshold ignores the robot’s current distance from the dock. The energy model computes exact cost for the current position, making the decision dynamic and position-aware.

Two-waypoint lookahead

One-waypoint lookahead guarantees the robot can reach WP_i and return, but not that it can continue to WP_{i+1} . The second lookahead closes this stranding trap.

PID at 10 Hz, not 1 Hz

The smoothing filter ($\alpha = 0.2$) is tuned for 10 Hz. At 1 Hz it would be nearly static, producing jerky or non-converging docking motion.

Learned overhead per session, not persisted

The overhead ratio reflects the current map and Nav2 planner at runtime. A stale value from a different session could be worse than the 1.2 bootstrap default.

DOCK mode never gives up

Skipping a patrol waypoint is recoverable. Failing to reach any dock leads to permanent battery death. Indefinite retrying is always preferable.

8. Results and Discussion

8.1. Parameter Tuning During Testing

Several parameters were revised after initial runs revealed instability and docking failures. Table 5 summarises every value that changed.

Table 5: Parameters revised during testing.

Parameter	Original	Final	Reason
safety_margin	15%	25%	Robot docking too close to empty battery
kp_lin	0.6	0.4	Velocity spikes caused Gazebo physics flip
kp_ang	1.2	0.8	Aggressive turns destabilised robot
max linear vel	0.15 m/s	0.10 m/s	Speed cap to prevent physics instability
PID rotation gain	0.7	0.4	Overshooting during Phase 1 alignment
dock contact dist	0.18 m	0.22 m	0.18 m threshold missed contact too often
smoothing α	0.3	0.2	Heavier damping (80/20) kills velocity spikes

8.2. Map 1 — Gazebo Default Map

8.2.1. Gazebo and RViz

Figures 6 and 7 show Gazebo and RViz during the same patrol run. Green spheres mark unvisited waypoints, grey visited ones, orange the docks, and the robot-mounted sphere reflects battery state. In RViz the Nav2 path, cylinder markers, and floating HUD panel are visible.

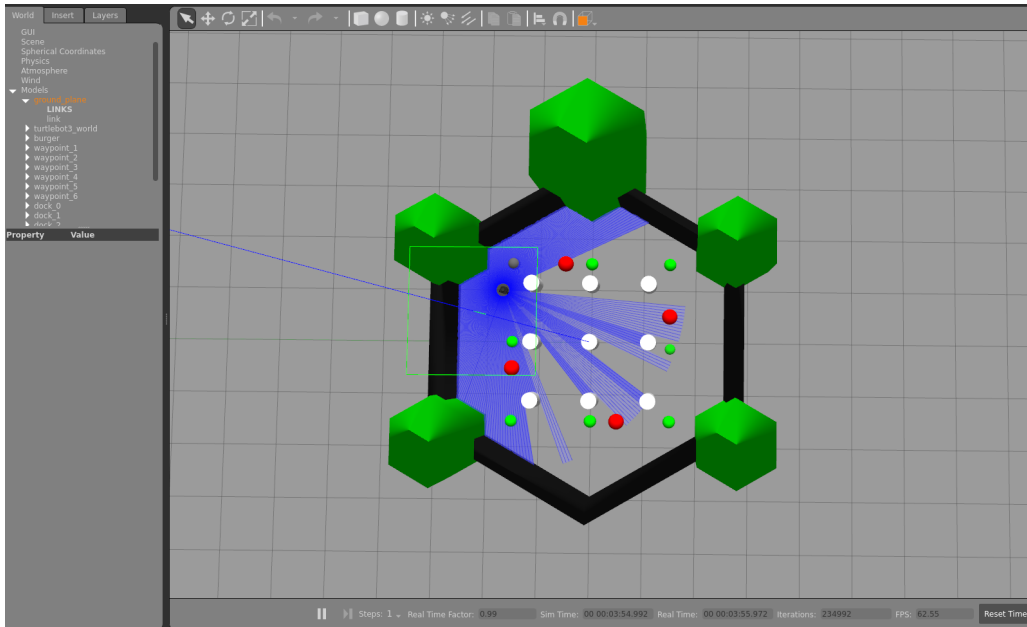


Figure 6: Gazebo mid-patrol: visited waypoint (grey).

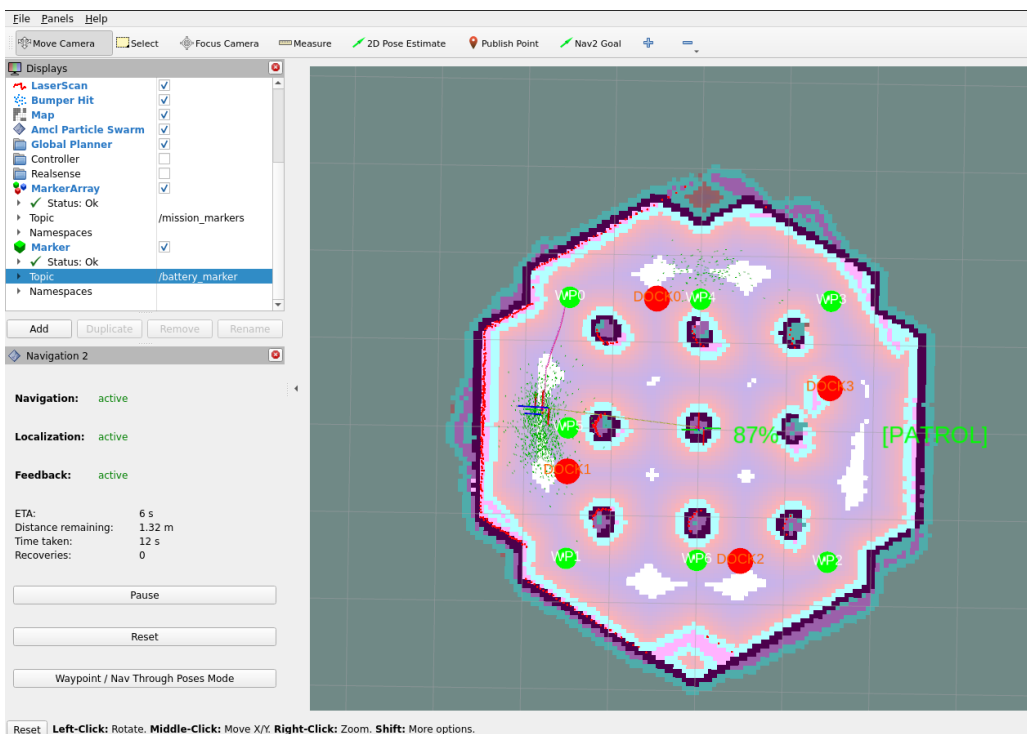


Figure 7: RViz: Nav2 path, battery indicator, and waypoint/dock markers.

8.2.2. Terminal Output

```
[INFO] [decision_node]: [ENERGY] wp1=17.4%  wp2=18.3%  direct=6.0%
                        battery=36.4%  overhead=0.76  margin=25.0%
[INFO] [decision_node]: [PATROL] WP5 -> (0.50, 1.50)
[INFO] [basic_navigator]: Navigating to goal: 0.5 1.5...
[INFO] [decision_node]: [PATH] 1.5_-1.8_to_0.5_1.5: real=4.46m
                        straight=3.47m  ratio=1.29
```

The worst-case scenario is $E_{wp2} = 18.3\%$, requiring $18.3 + 25 = 43.3\%$. Battery at 36.4% triggers the dock. The [PATH] line confirms Tier 2 learning: real path is 29% longer than straight-line — without this correction energy estimates would be 29% short.

8.2.3. RQT Graph

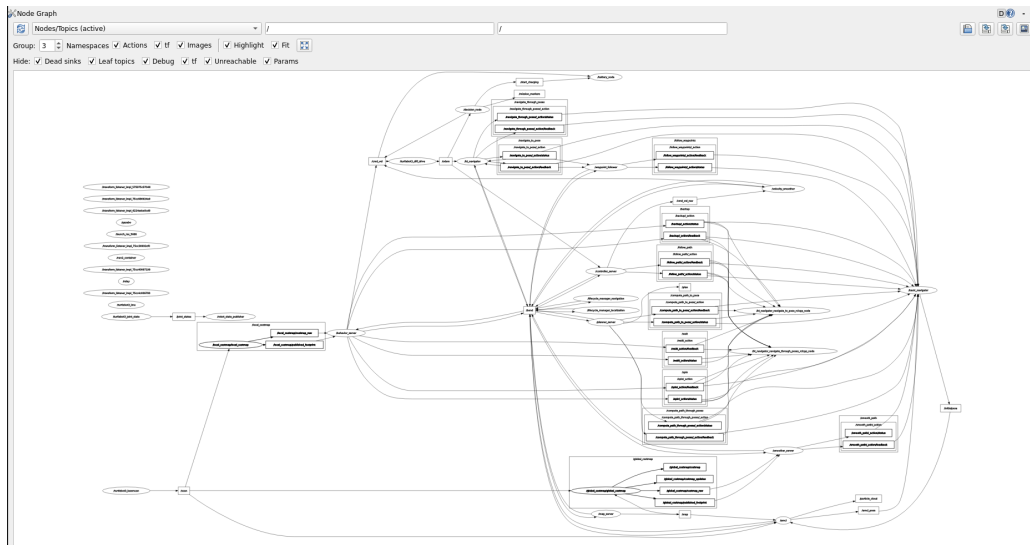


Figure 8: `rqt_graph`: nodes and topics active during Map 1 run.

8.2.4. TF Frame Tree

Figure 9 shows the TF frame hierarchy recorded during the run. The tree follows the standard ROS2 / Nav2 convention: `map` is the global fixed frame, `odom` provides local odometry, and `base_footprint` bridges to `base_link` — the physical robot centre. Five frames attach to `base_link`: the laser scanner (`base_scan`), inertial unit (`imu_link`), rear caster (`caster_back_link`), and both drive wheels. The `map`→`odom` transform is broadcast at 5.25 Hz by Nav2's AMCL localiser; `odom`→`base_footprint` at 29.6 Hz by the Gazebo odometry plugin.

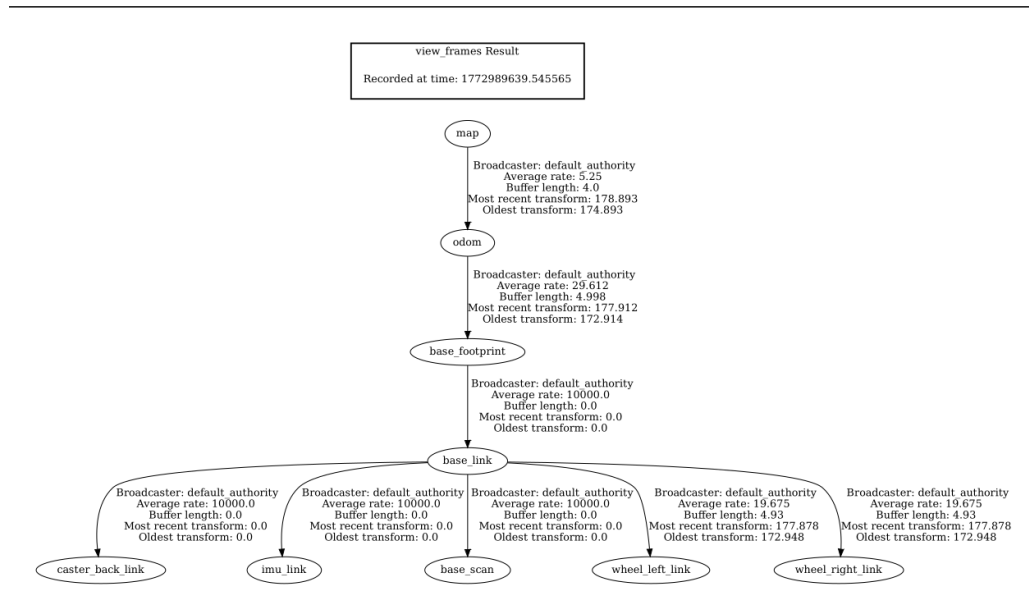


Figure 9: TF frame tree

8.2.5. Docking and Battery Trace

PID contact confirmed via terminal — robot transitioned to CHARGING at distance $< 0.22\text{m}$ from dock centre.

```

[INFO] [battery_node]:   Battery: 100.0%   <- mission start
...
[INFO] [battery_node]:   Battery:  36.4%   <- energy check triggers dock
[INFO] [decision_node]: [DOCK] Navigating to DOCK0 at (1.0, 0.0)
[INFO] [decision_node]: [DOCKING] Contact -- charging started
[INFO] [decision_node]: [CHARGING] Started. Target: 61.0%
...
[INFO] [battery_node]:   Battery:  61.3%   <- target reached, resuming patrol
[INFO] [decision_node]: [PATROL] Resuming from WP3
  
```

Charge target of 61% covers remaining waypoints plus the 25% margin — the robot did not charge to a wasteful 100%.

8.2.6. Problems and Compromises

Startup delay

A fixed 5-second delay is needed for Nav2, costmap, and odometry to initialise. A lifecycle-state check would be cleaner but the delay is reliable in practice.

Asynchronous path prewarm

Cache requests are non-blocking so the first departure may use Tier 2/3 estimates. The 25% margin absorbs this.

First-trip overhead underestimate

Learned ratio bootstraps at 1.2; on complex maps the real ratio may be higher until the first lap completes. Safety margin covers this.

Distance-based docking

Contact is a 0.22 m distance proxy — no physical sensor. Tuned manually and reliable

on flat Gazebo terrain.

Session-permanent blacklist

Blacklisted docks do not recover during a run. Not an issue in testing as dock failures were rare.

8.3. Map 2 — Custom Map

8.3.1. Gazebo and RViz

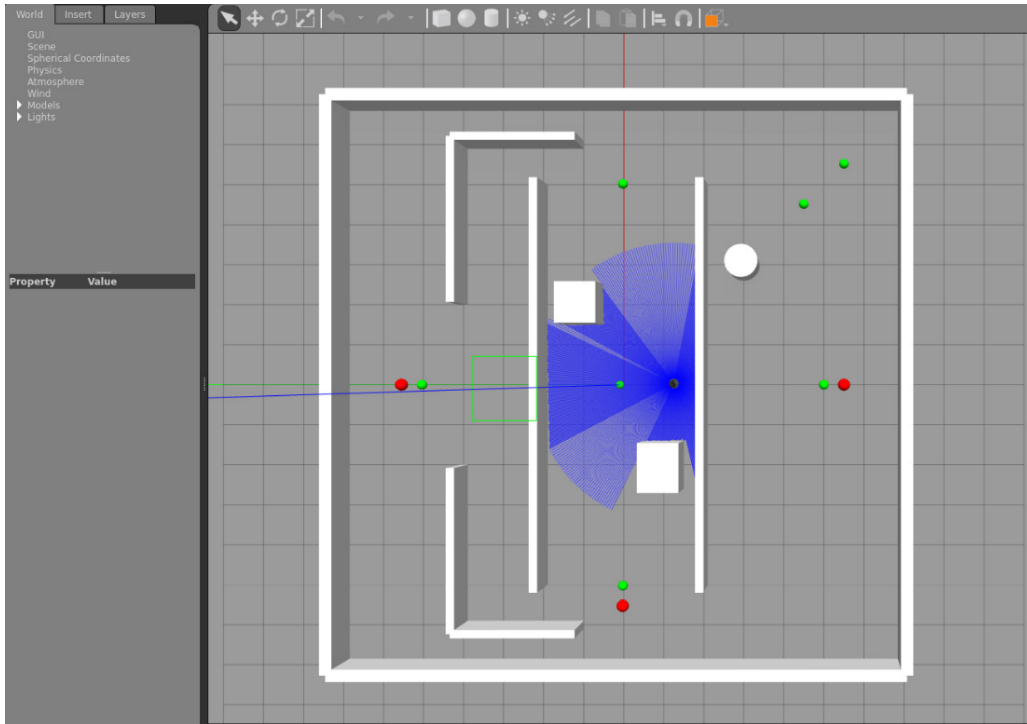


Figure 10: Gazebo view on the custom map.

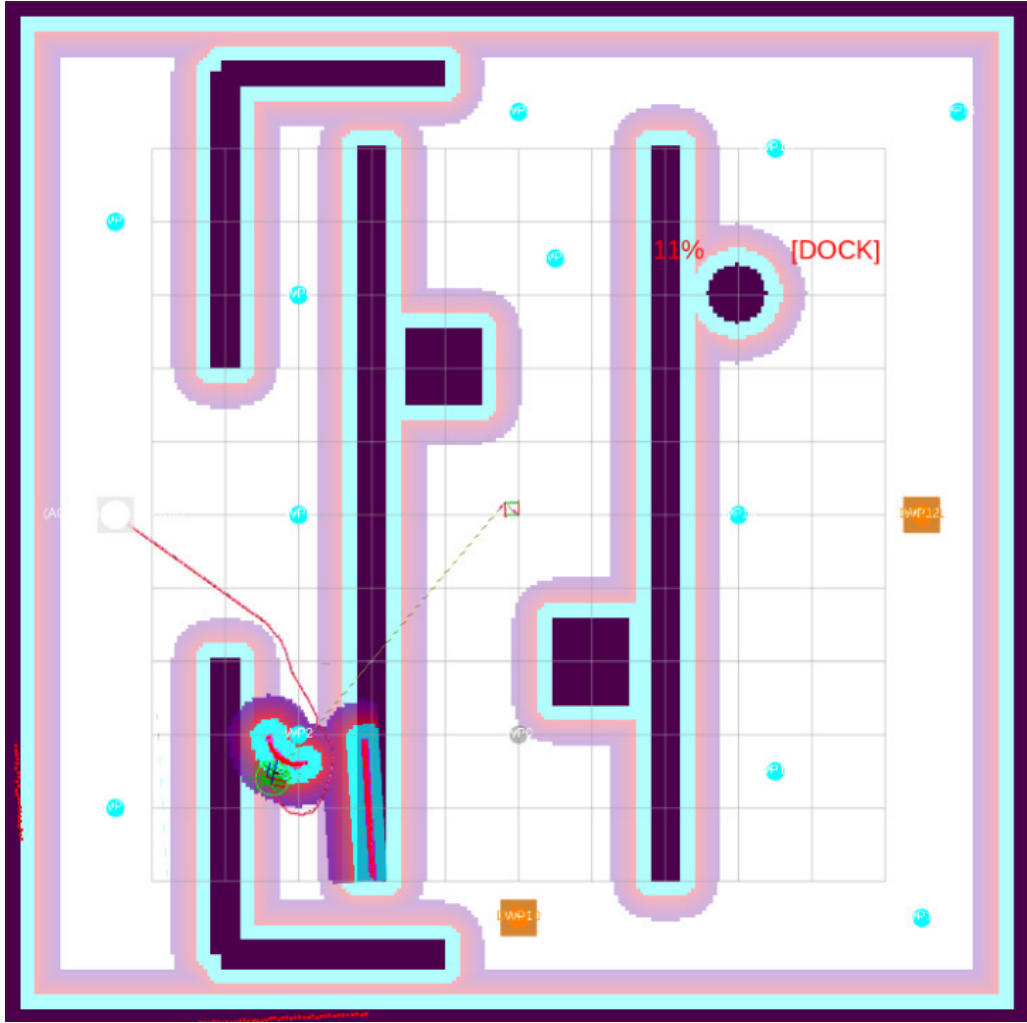


Figure 11: RViz display on the custom map.

8.3.2. Problems and Compromises

In addition to the general limitations described for Map 1, the custom map introduced map-specific issues:

Drain coefficient mismatch

The energy-per-metre constant $\gamma = 4.5\%/m$ was tuned on the default map. On maps with different obstacle density, Nav2 routes more tortuous paths - the robot travels further per straight-line metre than expected, causing the energy model to underestimate cost. If battery appears to drop faster than predicted, γ should be increased; if too slowly, reduced.

Overhead ratio convergence is map-specific

The learned ratio $\hat{r}$ starts at the bootstrap default of 1.2 on every run. On a heavily obstacle-packed map the real ratio may be 1.5 or higher, meaning the first few trips will underestimate energy until the ratio converges. On open maps the real ratio may be close to 1.0, making the bootstrap value overly conservative and causing premature docking.

Safety margin may need per-map tuning

The 25% margin was chosen conservatively for the default map. On maps where the

path overhead is consistently high and predictable, a larger margin may be needed to prevent low-battery situations. On simple open maps the margin wastes charging time. A per-map configuration option would be the correct long-term fix.

PID docking sensitivity to terrain

The 0.22 m contact threshold and PID gains assume flat, uniform terrain. On maps with tight corridors near the dock the robot may approach at an awkward angle and require more Phase 1 rotation cycles, increasing the time and battery consumed during docking.

9. Conclusion

This project implements a complete autonomous mobile robot mission system on TurtleBot3 / ROS2 Humble / Nav2. The core requirements - battery simulation, waypoint patrol, battery-triggered docking, and post-charge mission resumption - are fully satisfied.

The bonus predictive scheduling requirement is addressed through three-tier path estimation, two-waypoint energy lookahead, and adaptive charge targeting. Together these mechanisms prevent both premature and late docking, adapting automatically to the map's obstacle density as the learned overhead ratio converges over the course of a run.